Donghyung Lee

DevOps Engineer

E-mail: genius5711@gmail.com Blog(KR): somaz.tistory.com Blog(EN): somaz.blog GitHub: github.com/somaz94
Portfolio: somaz.blog/portfolio

Introduction

Hello, I am Donghyung Lee, a DevOps Engineer.

Aiming for stable infrastructure operation, I codify every change as IaC and write up every incident as a postmortem so it does not recur. Hybrid transitions and cloud migrations cut cloud costs 20-50%. My core strength is interrogating the purpose of each task and pursuing a better direction.

I have designed and built infrastructure across AWS and GCP public cloud and an OpenStack-based private cloud — standardizing container operations with Kubernetes, codifying infrastructure with Terraform and Ansible, and running GitOps deploy pipelines (GitLab CI, GitHub Actions, ArgoCD). For observability I covered all three signals — EFK logging, Prometheus/Grafana metrics, and OpenTelemetry distributed tracing. I automated repetitive work in Python and Bash.

I built in-house DevOps tools myself — git-bridge, a Slack APK bot, and more — and have also operated adjacent data-engineering and AI/ML ops through data-pipeline automation and an in-house GPU LLM service.

Across cloud and on-premises, I solely led the AWS + on-premises hybrid and the private cloud PaaS builds, and led the infrastructure side of the AWS → GCP migration (application-side migration in collaboration with the server dev team) — achieving 20% AWS and 50% GCP cost cuts (the latter by re-platforming Fargate onto GKE node pools plus an MSP discount), 67% faster deploys (30min → 10min), and replacing the manual build, deploy, and config-apply steps with the pipeline to eliminate human-error deployment failures.

I built an AWS EKS multi-cluster for a new game's cloud production launch and migrated helmfile deploys to a more stable auto-syncing model (ArgoCD pull-GitOps). The Argo Rollouts strategy moved from Blue-Green to Canary — removing the 503s that hit at promotion and letting backward-compatible patches ship without downtime — and, alongside unifying login via Keycloak central SSO, raised operational efficiency another notch.

I aim to keep contributing to infrastructure architecture advancement and organizational productivity.

Strengths

Asking 'Why' First
- Habit of asking 'why' first — define the business problem before picking the tool

Systematic Documentation & Knowledge Sharing
- Document and share every task, building organizational knowledge assets

Record-Based Continuous Improvement
- Postmortem-record every incident, infra change, and troubleshooting — preventing recurrence, driving improvement

Full-Stack Infrastructure Experience
- End-to-end infra across cloud (AWS, GCP), on-premises, networking, CI/CD, and monitoring

Skills

Cloud	AWS, GCP, OpenStack
Container Orchestration	Kubernetes, Docker, Cilium CNI, NGINX Gateway Fabric / Gateway API, Karpenter
Infrastructure as Code	Terraform, Ansible, Helm, Helmfile
CI/CD	GitHub Actions, GitLab CI/CD, ArgoCD, Argo Rollouts, Jenkins, GitOps
Monitoring & Observability	PLG Stack (Prometheus/Loki/Grafana), ELK Stack, EFK Stack, ECK Operator, Fluent Bit, Thanos, OpenTelemetry, Grafana Tempo
Security & IAM	Keycloak (Operator, OIDC IdP), Vaultwarden, GitLab SSO (OIDC), External Secrets Operator, Sealed Secrets
Automation	Go, Python, Shell Scripting

Database	MySQL, PostgreSQL, Redis, MongoDB
OS & Servers	Ubuntu, Rocky Linux, CentOS, Debian, GPU Server

Professional Experience

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

Sole DevOps infrastructure engineer at a game studio, designing, building, and operating AWS-based cloud and on-premises servers. Codified all infrastructure with Terraform/Ansible (100% IaC for new components) and documented operations via automation scripts and postmortems.Also achieved both cost optimization and operational stability through hybrid cloud architecture.

- Cut monthly infra cost 20% ($60,000 → $48,000) — sole owner of the AWS + on-prem hybrid design, 100% IaC on new components (Terraform/Ansible), role-separated envs for fault isolation
- Built the production-launch infrastructure for a new game — expanded a single on-prem dev cluster into an on-prem + AWS multi-cluster (new EKS, platform components 100% GitOps), Karpenter autoscaling across multiple spot families (to spread interruption risk) plus measurement-based request re-sizing that restored HPA scale-out and improved node bin-packing, keyless deploy via IRSA/Pod Identity · SSM (zero SSH keys), and now running real soft-launch traffic ahead of the 2026 Q4 global launch
- 90% faster log search (10min → 1min) and Metrics↔Traces↔Logs 3-signal observability in place — built an OpenTelemetry distributed-tracing pipeline (OTel Collector · Tempo, RED metrics generated from 100% of spans before sampling while storage is tail-sampled), 3-stage logging redesign ELK → EFK → ECK Operator (Fluent Bit HA · SQLite offset DB, zero log gap/duplication on Pod restart), kube-prometheus-stack for full-stack metrics
- Extended the EFK log pipeline into a product-analytics platform — ES Transform pivoting events to one row per user, Kibana serving D+1~D+30 user retention & cohorts, $0 analytics SaaS
- CI first-job bootstrap 5min → ~10s (shared toolchain image) — in-house K8s monorepo with GitOps wave-based auto-deploy (GitLab CI · ArgoCD · Jenkins), helmfile apply (push) → ArgoCD pull-GitOps migration (2 clusters), Canary via Argo Rollouts (production game), shell → Python automation migration (canonical templates)
- Platform standardization & security — migration consolidating ingress-nginx instances onto NGINX Gateway Fabric (single control-plane, wildcard TLS), central SSO unifying Harbor · ArgoCD · Vaultwarden on OIDC via Keycloak Operator, External Secrets Operator removing plaintext DB passwords
- Built & open-sourced internal productivity tools — Slack APK bot (90% faster QA delivery), GitLab ↔ Google Drive sync (80% less doc-management time), internal LLM (Ollama + Open WebUI, drove onboarding for 15 devs), Go OSS git-bridge · static-file-server, and Helm charts built during internal standardization released as OSS on ArtifactHub

Key Achievements

- 20% Monthly Infra Cost Reduction ($60,000 → $48,000, ~$144K annual savings → reinvested in new projects)
- Greenfield EKS Sole-Built Production Launch Infra (single on-prem dev cluster → on-prem + AWS multi-cluster, running real soft-launch traffic ahead of the 2026 Q4 global launch)
- 67% Deployment Time Reduction (30min → 10min, faster market response)
- 90% Log Search Time Reduction (per-Pod kubectl exec + grep avg 10min → single Kibana interface 1min)

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Led the large-scale AWS → GCP cloud migration at a game and blockchain startup, and built GitOps-based CI/CD pipelines.

- AWS → GCP large-scale migration (Shared VPC + Workload Identity Federation keyless GitHub Actions auth, subdomain-delegated DNS cutover during a maintenance window, Cloud Armor + Cloud CDN, self-hosted NFS replacing Filestore)
- Built GitOps CI/CD pipeline on GitLab CI · GitHub Actions · Jenkins · ArgoCD
- Automated certificates with Cert-Manager + Let's Encrypt
- Built game/blockchain data pipeline (MongoDB · CloudSQL · GA · Dune → BigQuery → Google Sheets, Cloud Functions + Scheduler + Dataflow, Terraform IaC)
- Built integrated monitoring on PLG Stack (Prometheus, Loki, Grafana)
- Designed & built GPU infrastructure for image-generation AI workloads — 10 in-house boxes, GCP A100, 20 external-IDC servers

Key Achievements

- 50% Cloud Cost Reduction (Fargate → GKE node pools + MSP discount, $10,000 → $5,000/month, ~$60,000 annually)
- 95% Data Collection Automation (0% data gap rate)

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Designed and built PaaS (Kubernetes + OpenStack + Ceph) private cloud infrastructure for financial and public sector clients, and led operations training myself.

- Customized PaaS solution design and construction (Kubernetes HA + OpenStack + Ceph)
- DP (Delivery Partner) Kubernetes operations training program design and delivery (6 months)
- Kubernetes certificate validity extension automation script (1yr → 10yr)
- OpenStack and Ceph storage cluster operations and optimization
- Regular cluster inspections and troubleshooting
- Ansible-based server provisioning automation achieving zero configuration drift
- Built and operated Jenkins-based build pipelines

Key Achievements

- 5 Cases Client PaaS Solution Deployments (Stable HA-based operation)
- 10x Certificate Renewal Cycle Extension (Annual → Once per 10 years)
- 50hrs Annual Ops Savings (Across 10 clients)

ETech System

2022.01 ~ 2022.04

System Engineer

- Hardware Server and SAN storage construction
- Transitioned to DevOps field for career growth

Opensource Projects

Kubernetes CRD

k8s-namespace-sync — Automatically sync Kubernetes namespace resources across clusters
helios-lb — Custom load balancer controller for Kubernetes
network-policy-generator — Kubernetes NetworkPolicy auto-generation controller

Helm Charts

nginx-gateway-cr — Tenant-level resource Helm chart for NGINX Gateway Fabric
elasticsearch-eck — Elasticsearch CR Helm chart for ECK Operator
kibana-eck — Kibana CR Helm chart for ECK Operator
certmanager-letsencrypt — Let's Encrypt resources Helm chart for cert-manager

GitHub Actions

Compress Decompress Action — Compress/decompress files in GitHub Actions workflows
Go Git Commit Action — Go-based Git commit creation and push
Multi Git Mirror Action — Automate mirroring across multiple Git repositories
Image Tag Updater — Automatically update image tags in YAML or Helm files
Kube Diff Action — Compare and report Kubernetes resource changes
Helm Chart Release Action — One-shot Helm chart release (package, gh-pages publish, OCI push)
Go Kubebuilder Test Action — kubebuilder operator test + manifests drift check
Helm OCI Push — Push Helm charts to OCI registries (GHCR, ECR, Harbor, Quay)
Helm Kustomize Lint Action — Lint and validate Helm charts (yamllint, helm lint, template, kubeconform)
Major Tag Action — Auto-update major version tags (v1) to the latest semver release
Go Changelog Generator — Generate changelogs from Conventional Commits

Ansible Galaxy

Ansible K8s IAC Tool — Automated installation collection for Kubernetes and IaC tools
Ansible Kubectl Krew — kubectl plugin Krew management role
Ansible User Management — Linux user and SSH key management role

DevOps Tools

bash-pilot — AI-powered Bash command assistant CLI tool
git-bridge — Git repository mirroring and sync CLI tool
static-file-server — Go-based static file server with directory UI, file preview, search, ZIP batch download
kube-diff — CLI to diff Kubernetes manifests (YAML, Helm, Kustomize) against a live cluster
kube-events — CLI to view and summarize Kubernetes events with grouping, filtering, colored output

Chrome Extensions

Dev Toolkit — A collection of useful developer utilities
QRify — Quickly generate QR codes for URLs or text

Education

Chungnam National University - Public Administration (Transfer)	2018.03 ~ 2020.08
National Institute for Lifelong Education - Business Administration (Academic Credit Bank)	2016.09 ~ 2018.02

Certifications

Engineer Information Processing	2021.11
AWS Certified Solution Architect - Associate (Expired)	2021.11
Network Administrator Level 2	2021.10
Linux Master Level 2	2021.10
Computer Specialist Level 1	2021.04
TOEIC Speaking Test - 130/Intermediate Mid 3 (Expired)	2021.02

Study

Tech blog (somaz.blog, English) — DevOps · Kubernetes · IaC writing for global readers	2025.01 ~ 현재
Tech blog (somaz.tistory.com, Korean) — ongoing DevOps · Kubernetes · IaC learning and operations notes	2023.04 ~ 현재
T101(Terraform) Study	2023.08 ~ 2023.10
AEWS(EKS) Study	2023.04 ~ 2023.06
PKOS(Kubernetes) Study	2023.01 ~ 2023.02
Cloud Security Training	2022.02 ~ 2022.04
Security Solution Operation Specialist Training Program	2021.07 ~ 2021.12

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- AWS EKS, GCP GKE production operations & cost optimization (20% reduction via hybrid transition, 50% via GCP migration — re-platforming Fargate onto GKE node pools plus an MSP discount, savings reinvested in new projects)
- Solely built and launched the greenfield EKS production environment for a new externally-published game — two Karpenter NodePools split by workload — ARM multi-family spot for CI builds, on-demand for the game workloads, keyless authentication via IRSA / Pod Identity, platform components managed 100% declaratively via GitOps, soft launch opened 2026.07
- Terraform IaC infrastructure automation & version control
- Built and operated GPU infrastructure for AI workloads — a two-A100 training server on GCP as Terraform IaC (NerdyStar), and an in-house LLM inference service on Kubernetes with Ollama and Open WebUI on an on-prem GPU box (Phantom)

CI/CD Pipeline & GitOps

Maximizing development productivity through GitOps-based deployment automation

- CI/CD pipeline design & enhancement with GitHub Actions, GitLab CI, Jenkins, ArgoCD
- 67% deployment time reduction with continuous GitOps deployment shortening Time to Market, 95% rollback time decrease (30min → 1-2min)
- Moved the production game service from Blue-Green to Argo Rollouts Canary — removing the 503 race where healthy targets momentarily dropped to zero at promotion, so backward-compatible patches ship with zero downtime

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- Evolved from ELK → EFK Stack, then redesigned into ECK Operator-based Elastic Stack 9.0 CRD declarative management (automatic TLS rotation, automated rolling upgrades)
- kube-prometheus-stack with custom alert rules, Grafana dashboards, and DB exporters (MySQL, Redis, Elasticsearch, PostgreSQL) integrated

- Added distributed tracing with OpenTelemetry + Tempo to complete the Metrics ↔ Traces ↔ Logs picture — RED metrics generated from 100% of spans before sampling to keep accuracy, with tail sampling applied only at trace storage to cut cost

Automation & Team Enablement

Operations automation tooling with Python and Bash, plus training and standardized materials that let client ops teams run things themselves

- Migrated shell → Python (stdlib only) + wave-based component auto-deploy + auto-upgrade MR generation
- Established self-sufficient Kubernetes ops at DP (Delivery Partner) sites — designed and ran a 6-month training program (avg 10 attendees, 4h/session), standardized materials reused across other clients
- The same discipline outside work — built a pipeline that collects, quality-checks, and republishes from 10+ public APIs, running on an automated twice-daily weekday schedule (economy dashboard), alongside a backend-free diagram editor published and run in the open

Kubernetes Platform Modernization & Open-Source Contribution

Migration of network/logging stacks, releasing reusable public charts as open source, and contributing upstream

- Migrated multiple ingress-nginx instances to a single NGINX Gateway Fabric control-plane + per-class Gateway CRs (full cutover, wildcard TLS consolidated)
- Released open-source Helm charts (nginx-gateway-cr, elasticsearch-eck, kibana-eck) alongside Kubernetes CRD controllers, composite GitHub Actions, and Ansible collections on ArtifactHub / GitHub Marketplace / Ansible Galaxy
- Contribute upstream when a feature is missing — added Gateway API HTTPRoute support to Helm charts in apache/airflow, OpenTelemetry, and prometheus-community, plus bug fixes to Go projects

Infrastructure Security & IAM

Centralized in-house authentication and secret governance via Keycloak Operator-based SSO and Vaultwarden password management

- Introduced central SSO with the Keycloak Operator + KeycloakRealmImport — let existing GitLab accounts keep logging in, integrated ArgoCD / Harbor / Vaultwarden OIDC (verification checks PASS), and established a single authorization flow from GitLab groups → Keycloak mappers → ArgoCD roles
- Deployed Vaultwarden on Kubernetes via Helm with GitLab SSO + automated backups (scheduled SQLite dumps) + interactive restore script — centralized management of internal secrets

Professional Experience

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As the sole DevOps infrastructure engineer at a game development company, I design, build, and operate AWS-based cloud and on-premises servers. The dev and qa servers are on-premises, while review and prod are on AWS, operating a hybrid architecture that reduced monthly infrastructure costs by 20% ($60K → $48K); the savings were reinvested in new project infrastructure.
Currently providing secondary technical support and operations for a live-service game in collaboration with the external publisher, and building on-premises/AWS infrastructure to improve development productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution: hybrid architecture design & build 100% (solo); live-game secondary technical support & operations in collaboration with the external publisher's ops team 2024.10 ~ Present (In Operation)

Problem

While operating the entire infrastructure on AWS, high cloud costs of $60,000/month necessitated cost optimization.

Approach

Workload-specific cost analysis revealed that the dev/qa environments had minimal traffic variation, making cloud auto-scaling unnecessary, while only review/prod required scalability and stability. Designed a hybrid architecture transitioning dev/qa to on-premises while keeping review/prod on AWS.
The two environments are operationally independent without VPN (application traffic isolated); the image registry is also split by environment (dev/qa = Harbor, review/prod = ECR), with each environment building independently to match its architecture. Unified operational standards through Terraform and Ansible IaC integration (100% IaC coverage for new components) across both environments, achieving consistent infrastructure management without increasing operational complexity. For IAM, only service and CI identities are managed as code and human identities stay outside it; the developer accounts that need console access are the exception, designed so Terraform issues no access key — each person creates and rotates their own — keeping long-lived credentials out of the state file.

Troubleshooting

- When configuring ALB Ingress on EKS, needed to route to different backends per game client version.
  Resolved by combining custom header-based routing rules with Target Groups in ALB conditional routing
- During rolling deploys of the ArgoCD-managed production services, the ALB target deregistration delay didn't line up with Pod SIGTERM handling, dropping some session connections. Achieved 0 connection drops via 3 changes — (1) aligned ALB drain timing: a preStop hook + readiness-first fail block new requests during termination while in-flight requests are served to completion, (2) tuned the RollingUpdate strategy surge-first to fit replica scale, holding capacity through rollout, (3) applied a PDB to multi-replica services so Karpenter consolidation never evicts Pods simultaneously.
  Verified on a CloudWatch-backed Grafana dashboard that separates ELB 5xx from Target 5xx, comparing target-group healthy/unhealthy host counts and p95 response time before and after each deploy

Results

- Workload analysis-based infra optimization reducing monthly costs by 20% ($60,000 → $48,000), ~$144,000 annually — the savings were reinvested in new project infrastructure
- Ensured operational stability and fault isolation through environment role separation

---

Project 2: CI/CD Pipeline Construction & Enhancement

Contribution 100% (Full infra & pipeline ownership) 2024.12 ~ Present (In Operation)

Problem

Developers manually built and deployed to servers, taking an average of 30 minutes per deployment with frequent failures due to human errors.

Approach

Built a GitOps-based automated deployment environment combining GitLab CI and ArgoCD. A developer only has to Git Push and build→test→deploy runs automatically, while ArgoCD keeps watching cluster state and converges it back whenever it diverges from what Git declares.
Image builds run on Kaniko: building with a Docker daemon in CI would require a privileged container on the runner (Docker-in-Docker), and Kaniko builds without one — hence the choice. Multi-environment (dev/qa/review/prod) deployments are managed from a single pipeline.

Troubleshooting

- One bad commit could delete whole child Applications → split automatic deletion (`prune`) by layer. Workload Applications run `prune: true` + `selfHeal: true` so drift reverts immediately, while only the app-of-apps root and the AppProject layer run `prune: false`, leaving deletions for a human.
  Self-healing stays on at the root too, so a hand-deleted child app comes back. Every sync event goes to Slack for traceability
- Moved the data-upload path to keyless — deploys used to SSH straight into the target instance and push files (scp/rsync), which meant the deploy account had to hold an SSH key.
  Removed that path: files are staged into an S3 transit bucket and the target instance pulls them down itself via SSM SendCommand (remote execution without SSH), eliminating SSH keys entirely and removing key-leak risk

Results

- 67% deployment time reduction (30min → 10min)
- Cut the source-fetch step of data deployment from ~139s to ~30s — replaced a plain `git pull` with a fetch that takes only the latest commit and defers file contents until they are actually needed (shallow + blobless fetch)
- Established merge-driven continuous deployment through GitOps automation — roughly 100 deploys/week per core service repo, cutting the time a new feature takes to reach users
- Replaced the manual build/deploy/config-apply steps with the pipeline to eliminate human-error deployment failures, and the cross-repo MasterData pipeline auto-fans-out to downstream services on data changes while preventing circular (loop) triggering

---

Project 3: Central Logging & Observability Platform (Collection → Integrity/HA → Product Analytics)

Contribution 100% (Sole design across the full lifecycle — collection · integrity · product analytics) 2024.11 ~ Present

Starting from a log-scattered on-premises environment, advanced centralized logging through three stages (ELK → EFK → ECK Operator), hardened collector data integrity / HA, then repurposed the accumulated EFK log pipeline into a game user-retention analytics platform. Solely designed and operated the full observability lifecycle — build → operate → improve → analytics.

3-1. Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement, 2024.11 ~ Present)

Problem

Server and container logs were distributed across the on-premises environment, making root cause analysis time-consuming during incidents, with no unified monitoring system in place.

Approach

Enhanced the logging stack in **three stages: ELK → EFK → ECK Operator**. The initial ELK collected logs with a per-node Filebeat DaemonSet and processed them in a central Logstash tier; as the JVM-based Logstash tier grew memory-hungry, migrated to EFK — collection by a lightweight Fluent Bit DaemonSet, processing by Fluentd.
As Elastic's official Helm chart stalled for over two years, moved to CRD-based declarative management (ECK Operator), performing a major upgrade of Elastic Stack from 8.5.1 to 9.0.0, and templated the on-premises EFK stack into AWS(EKS) `-aws` stacks to port the same model — automating log lifecycle (ILM) and S3 snapshot retention with keyless IRSA.

Troubleshooting

- Auto-tracking Elastic Stack releases on the on-premises stack pulled a version present in the artifact feed but whose image was not yet published → `ImagePullBackOff`, and rolling back was refused by the ECK admission webhook (`Downgrades are not supported`), blocking the rollback path and killing the operator.
  Judging **being unable to undo** the real risk rather than breaking, added an image-publication check, a health pre-flight, a snapshot warning on major bumps, and a recovery path accounting for the webhook

Results

- 90% log search time reduction (per-Pod access → single Kibana interface), improved MTTR keeping service availability stable
- Redesigned from heavyweight Logstash to a lightweight Fluent Bit + Fluentd architecture, and the ECK Operator migration removed manual operations for TLS certificates, accounts, and upgrades (rolling upgrades via a single-line CR spec.version change), completing a stalled 2-year Elastic Stack major upgrade (8.5.1 → 9.0.0)

3-2. Fluent Bit Log Collector Data Integrity & HA Hardening (2026.03 ~ 2026.05)

Solved log re-index/loss risk from Fluent Bit tail-offset loss on Pod restart with a SQLite offset DB — a DaemonSet runs one Pod per node, so the offset lives on a per-node local path whose lifetime matches the node's, giving zero re-index / zero duplicates, with alerts for a collector that stops quietly — which surfaces as missing logs, not as an error

---

3-3. Game User Retention Analytics Platform (EFK Logs → Product Analytics, 2026.06 ~ Present)

Extended a search-only EFK stack into a product-analytics platform via an Elasticsearch Transform (5-min) pivoting events to one row per user — a Kibana dashboard delivering D+1~D+30 retention and cohorts, no separate SaaS

Project 4: Developer Productivity Tools (APK Deploy Bot · Doc Automation · LLM Service · Git Mirroring · Static File Server)

Contribution 100% (Full system design & development) 2025.02 ~ Present

Designed, developed, and operated 5 internal tools to automate repetitive manual tasks and boost development team productivity.

4-1. Slack APK Distribution Automation Bot (4 weeks, 2025.02)

Deployed a Python bot on Kubernetes auto-sending completion messages and download QR codes to Slack on Jenkins APK build finish — 90% QA delivery-time reduction, version confusion eliminated

---

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.06)

One-way sync of design documents from GitLab into a Google Drive shared drive via rclone, triggered on push — comparing on `--checksum` so only changed files transfer — 80% doc-management reduction (10min→2min), GitLab held as the single source of truth with direct Drive edits overwritten on the next sync

---

4-3. Internal LLM Service Deployment (4 weeks, 2025.11)

Ran an in-house LLM on an on-prem GPU server with Ollama and Open WebUI on docker compose — a coding model kept resident and opened to the dev team

---

4-4. Git Repository Mirroring Tool + Retry API Follow-up (4-week initial build, 2026.02 ~ 2026.06)

Built and open-sourced git-bridge, a Go bidirectional Git mirroring tool, to end the gaps and lag of manual CodeCommit/GitLab/GitHub syncing — 100% automated across 10 repos and ~30 syncs/day, with a Retry API that recovers without exposing the webhook secret and `ref_overrides` direction-pinning that blocks history rewinds (6/6 PASS on a test-only repo)

4-5. Static File Server - In-house Development (Open Source, 2026.03 ~ Present)

With halverneus/static-file-server offering no directory UI, search, or ZIP batch download and its upstream maintenance stalled, built a Go static file server in-house, released it under Apache-2.0, and deployed it via an in-house Helm chart on NFS as a full replacement — handling

~30 downloads/day and 5 APK releases/week

Project 5: Kubernetes Cluster Operations Enhancement

Contribution 100% (Solo design — monitoring · upgrade automation · Helm management framework · NGF migration · OSS Helm charts · storage performance optimization · Shell→Python migration; captured as automation scripts establishing a standard procedure → so the team can reuse)
2025.12 ~ Present

To keep the cluster running reliably, overhauled monitoring, automated K8s upgrades and Helm management, and migrated the network controller onto the Gateway API.
Released the artifacts from the NGF migration and ECK Operator adoption as OSS Helm charts, resolved storage bottlenecks via control-plane / DB-node SSD migration and build I/O isolation, and modernized the infra deploy/upgrade pipeline with wave-based automation + a Shell → Python migration.

5-1. Monitoring & Alerting Enhancement (2025.12 ~ Present)

Designed custom alerts and Grafana dashboards on kube-prometheus-stack, integrating DB, node-exporter, and Cilium metrics — custom alert rules grouped by component, with severity routing and inhibit rules removing duplicates

---

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ Present)

Scripted the Kubespray upgrade procedure and a nine-item post-upgrade check (nodes, etcd, API server, certificate expiry and more), standardizing version management with a Helm framework and a canonical-template synchronizer — 90% verification-time reduction, with certificate renewal scripted to remove expiration outages

---

5-3. Ingress-nginx → NGINX Gateway Fabric (NGF) Migration (2026.04)

Migrated the on-prem ingress-nginx fleet to a single NGF 2.x control-plane + per-class Gateway CRs — MetalLB IPs preserved so DNS/firewall unchanged, incident-free cutover (30-60s per class), one wildcard cert cutting manual renewal 50%

5-4. Open-Source Helm Charts — nginx-gateway-cr · elasticsearch-eck · kibana-eck (2026.04 ~ 2026.05)

Refined NGF and ECK outputs into reusable charts (nginx-gateway-cr·elasticsearch-eck·kibana-eck), published on ArtifactHub under Apache-2.0 — cut the mgmt cluster over to public OCI charts with cluster_uuid unchanged

---

5-5. Storage Performance Optimization — etcd / DB SSD Migration + Build I/O Isolation (2 weeks, 2026.04 ~ 2026.05)

Diagnosed GitLab Runner buildx's disk-I/O spike behind second-scale etcd and game-DB latency on HDDs, migrated the control-plane and DB worker to SSD, and isolated buildx onto a dedicated build VM on separate hardware — game DB COMMIT recovered from 9~14s to the ms range, zero play-blocking incidents during builds

---

5-6. Infra Deploy/Upgrade Automation + Shell → Python Migration (2026.03 ~ Present)

Problem
I had built and run the internal Kubernetes infra monorepo's automation myself on helmfile + shell, but as components piled up, shell + awk could no longer handle per-environment branching or complex YAML edits, and there was no way to test any of it.
New deploys, chart upgrades, and template sync were all still manual.

Approach
Structured the CI pipeline into 6 waves (bootstrap → core → security → db-redis → observability → apps), detecting MR diffs to deploy only the changed components via helmfile, with a human approving before anything reaches production. Automated upgrades too: when a new chart version lands, CI opens an MR laying out what changed, the config differences, and any compatibility-breaking changes.
Moved the existing shell + awk automation to Python (stdlib only), splitting standard templates, sync/backup, and CI scripts into modules with per-component upgrade modules and unit tests (one test file per module, run by `unittest discover` with no external runner). Adopted Project 7's helmfile-tools image in CI.

Results

- **Wave-based sequential component deploys + auto-upgrade MR generation**, slashing the operational burden of infra deploys and upgrades
- **Migrated the shell + awk automation to Python** — removing the walls of per-environment branching and complex YAML edits while gaining readability and testability.
  Built on the standard library alone with no external packages, so it runs anywhere python3 exists, with no install step or virtualenv
- Local (macOS venv) and remote (CI container) execution share the exact same code, removing environment-drift issues between dev and deploy

5-7. helmfile apply (push model) → ArgoCD Pull-GitOps Migration (App-of-apps Control Plane, 2026.06 ~ Present)

Problem

Platform releases were pushed straight into the cluster from the CI runner via `helmfile apply`, tying the deploy actor to the runner.
As the estate expanded to multiple clusters, a pull-based GitOps control plane was needed to declaratively converge per-cluster drift.

Approach

Migrated the CI runner's `helmfile apply` push deployment to ArgoCD pull-GitOps. A Matrix + Git-files generator ApplicationSet reads each component's `argocd/*.yaml` marker files to create Applications, registering the on-premises and AWS platform releases across both clusters.
Existing helm releases were handed over to ArgoCD without re-creation (in-place adoption) with deploy order preserved, and App-of-apps prune-cascade guardrails plus least-privilege AppProjects shrink accidental mass-deletion risk and blast radius.

Results

- Migrated from `helmfile apply` push deployment to ArgoCD pull-GitOps, managing the platform releases across the on-prem and AWS clusters from a single declarative control plane, adopting existing helm releases without re-creation and preserving deploy order
- App-of-apps prune-cascade guardrails eliminate the risk of mass GitOps deletion, and least-privilege AppProject whitelists shrink blast radius

---

Project 6: Infrastructure Security System

Contribution 100% (Design, deployment, SSO integration) 2026.04 ~ Present

Building out the security infrastructure for operations in sequence — secret management, central authentication, and remote access.

6-1. Vaultwarden Password Management System (1 week, 2026.04 ~ Present)

Centralized internal secrets scattered across Slack/email/personal vaults with Vaultwarden — deployed on Kubernetes via Helm with GitLab SSO (OIDC) and RBAC, at zero SaaS cost

6-2. Keycloak Operator-based Central SSO Introduction (2026.04 ~ Present)

Problem

Internal tools — ArgoCD, Harbor, Vaultwarden — each wired GitLab directly as their OIDC IdP, hard-coding the identity source into every tool.
Because GitLab offers OIDC only as a side feature, it could not front an external identity source such as LDAP or carry per-client auth policy, and swapping the source meant reconfiguring each tool one by one.
Introduced a login broker (Keycloak) to decouple the tools from the identity source.

Approach

Introduced the Keycloak Operator + KeycloakCR + KeycloakRealmImport with a PostgreSQL backend to broker GitLab login, so existing accounts kept logging in. Codified Realm / Client / IdP / Group / Mapper setup via a bootstrap script and migrated Harbor, ArgoCD, and Vaultwarden's OIDC from direct-GitLab to via-Keycloak.
For RBAC, GitLab groups flow through Keycloak group mappers into ArgoCD role mappings, consolidating the permission SSOT onto Keycloak.

Results

- **ArgoCD, Harbor, and Vaultwarden OIDC integrations completed with verification checks PASS** — removed direct dependence on GitLab
- Consolidated the GitLab groups (`server`, `global-admin`) → Keycloak group mappers → ArgoCD roles authorization flow onto Keycloak as the single SSOT, laying the groundwork for future LDAP federation, MFA enforcement, and central session policy
- Documented the pitfalls hit during adoption (IdP providerId / scope / mapper config / Operator idempotency / Harbor user mapping) in the plan doc to prevent recurrence on future cluster rebuilds

6-3. GitOps Secret Management — External Secrets Operator · Sealed Secrets (2026.05 ~ Present)

Replaced plaintext DB passwords in Git by wiring External Secrets Operator to AWS Secrets Manager for ExternalSecret references and standardizing the Harbor pull secret as a cluster-wide SealedSecret — making AWS SM the single source of truth

6-4. OpenVPN Remote-Access Server (2026.07)

Built an OpenVPN remote-access server in-house on a physical server, standardizing internal-network remote access at up to 50 concurrent connections — stood up an in-house PKI with easy-rsa issuing a distinct EC certificate (prime256v1) per client, with tls-crypt encrypting the control channel so attempts without a valid certificate are rejected during negotiation (traffic on AES-256-GCM / TLS 1.2+).
Full-tunnel by default with split-tunnel switchable, and install plus certificate issue/revoke automated behind idempotent scripts

Project 7: Shared Toolchain Image Layered Architecture + Auto Version Cycle

Contribution 100% (Architecture design · CI standardization · automation pipeline) 2025.09 ~ 2026.05

Problem

Across the repos in the GitLab CI standardization scope, each job re-installed helm / helmfile / kubectl / crane / aws-cli on top of an Alpine base, making first-job bootstrap take an average of ~5 minutes. Tool versions were scattered across the consumer repos, so adding a new version effectively required a coordinated bump across every consumer repo.
Reliance on `:latest` tags also undermined reproducibility and made CVE tracking hard.

Approach

Layer 1 (Mirror): copied external registry images (Docker Hub, GHCR) into the internal Harbor via `crane copy`, preserving multi-arch manifests.

Layer 2 (Custom): built fat images on top of Layer 1 (helmfile-tools, base-tools, node-build, rclone-backup, aws-cli-tools) so consumer repos get the tools immediately on image pin alone.

Tier 1~2 SSOT: introduced a shared `.toolchain_build_custom` template + central `TOOLCHAIN_*_TAG` variables so consumer repos no longer track image tags directly.

Shared CI template SSOT: extracted the CI jobs that had been copy-pasted per repo into a central shared CI template repo, with each repo referencing them via `include`. Manages the auth (keyless AWS auth) / build (kaniko) / deploy (ArgoCD) / backup (Google Drive) job templates plus shared anchors (git·ssh·rules·job-config·packages) from a single source, eliminating copy-paste drift.

Automated version cycle: a weekly schedule runs (1) to detect new releases, and once an operator starts the pipeline, (2)-(4) follow as auto-MRs. Human judgment is required at exactly two points: starting the pipeline and merging the Tier 2 MR:

- (1) Upstream version detection
- (2) Layer 1 mirror auto-MR
- (3) Layer 2 rebuild + Tier 2 TOOLCHAIN_*_TAG sync auto-MR
- (4) Auto-propagation to the consumer repos (excluding the template provider repo)

Troubleshooting

- While auto-tracking new tool versions, helmfile 1.5.1 began requiring helm >= 3.18.6 while helm stayed pinned at 3.16.4, breaking the `build_custom` build → since cross-tool compatibility requirements only surface in upstream release notes and cannot be auto-detected, introduced a minor-guard for helm / helmfile / kubectl: only patches within the current minor bump automatically, while minor/major bumps are split out for a human to verify compatibility first (overridable once via `*_TARGET_MINOR` env vars for a planned minor bump)

Results

- **First-job bootstrap time cut from 5min to ~10s (~95% reduction)**, with 100% tool-version consistency across the consumer repos, compressing the scattered version points into a 4-tier governance flow (ARG → central var → consumer indirection → lint drift check)
- Automated new-version propagation for helm / helmfile / kubectl / crane / rclone, while gating the high-blast-radius image rebuild behind human review rather than the schedule — narrowing the human decision points to two
- Built a risk-tiered auto-upgrade pipeline across components (T1 weekly / T2 biweekly / T3 monthly + pre-flight) — version detection → auto-MR → Slack alert → wave-sequenced apply, with a MAJOR_PIN guard and T3 atomic rollback — and built amd64 + arm64 multi-arch images via docker buildx while mirroring external images into Harbor with crane, removing external-registry dependency

---

Project 8: AWS EKS Production Game-Launch Platform (New Externally-Published Title)

Contribution 100% (Solo greenfield EKS design & build) 2026.06 ~ Present (Building & Operating)

Problem

Separately from the live game's estate (AWS review·prod), the new externally-published game ran only as a single on-premises development cluster, and launching it to production required a separate, scalable, highly-available AWS EKS production environment — on-premises alone could not handle global traffic and autoscaling needs.

Approach

Built a greenfield EKS cluster in eu-central-1, expanding the estate into an on-premises dev + AWS production multi-cluster hybrid, and managed the platform components entirely via GitOps (ArgoCD).

Autoscaling / networking: Chose Karpenter over Cluster Autoscaler and split the NodePools by workload — ARM multi-family spot for CI builds (more independent capacity pools, so fewer builds reclaimed mid-run) and on-demand for the game workloads, while AWS Load Balancer Controller (ALB/NLB) + ExternalDNS (Route53) + a wildcard ACM cert automate ingress, DNS, and TLS.

Logging / secrets: an eck-operator-based EFK stack (Elasticsearch 3-node 3-AZ, EBS gp3, S3 snapshot SLM) runs HA, with External Secrets

Operator externalizing secrets; every ServiceAccount authenticates via IRSA / Pod Identity, removing static keys.

Auth / deploy: ArgoCD integrates GitLab OAuth SSO + game-scoped RBAC; client artifacts ship via Jenkins → S3 + CloudFront, and server manifests reach the bastion's EFS mount via an S3 transit bucket + SSM SendCommand — the bastion is discovered dynamically by Name tag, so deploys use no SSH keys.

Troubleshooting

- Game workloads must not lose a node mid live session or mid Apple review, so they run on a dedicated on-demand NodePool → consolidation is left on, not blocked.
  The app is stateless HTTP (sessions in Redis), so a preStop drain + a PDB of `minAvailable: 1` + a disruption budget of `nodes: 1` drain one node at a time and never drop below 1 Ready. `karpenter.sh/do-not-disrupt` is unused — it would block consolidation entirely
- Nodes grew on every rollout restart and never shrank → root cause was Karpenter's consolidation policy interacting with the rolling-update `maxSurge`, leaving newly added nodes un-reclaimed.
  Measured per-node memory-request utilization (92% on one node vs 35% on another), traced it to one workload's over-sized request, cut it 2Gi → 512Mi (measured usage ~101Mi) and changed the consolidation condition alongside it, restoring bin-packing
- During rolling deploys, ALB routed traffic to not-yet-ready Pods → resolved by combining ALB Pod readiness gate + PodDisruptionBudget + graceful drain

Results

- Expanded from a single on-premises dev cluster to an on-premises + AWS production multi-cluster hybrid, establishing the cloud production launch foundation for the new externally-published game
- Managed the platform components 100% declaratively via GitOps (ArgoCD), and established static-key-free authentication with IRSA / Pod Identity + SSM SendCommand keyless deployment, removing bastion SSH-key leak risk
- Widened the spot NodePool across multiple ARM instance families to increase the number of independent spot capacity pools — lowering the aggregate interruption probability so fewer in-flight builds get reclaimed, with instance size pinned so the per-node cost shape is unchanged.
  Production and review workloads share one NodePool rather than two, so they bin-pack together — splitting them would impose a one-node floor per pool
- ArgoCD app markers plus a production CI/CD transition doc make this a solo build the team can still operate and rebuild by the same procedure — a documented hand-off path

---

Project 9: Observability Hardening — Distributed Tracing & Zero-Downtime Delivery

Contribution 100% (Solo design & build — OTel tracing pipeline · Argo Rollouts Canary · soft-launch operation) 2026.06 ~ Present

On top of the AWS EKS production game-launch platform (Project 8), strengthened observability and delivery for the new externally-published game's soft launch.
Filled the missing 'traces' axis of the logging/metrics-only observability platform with an end-to-end OpenTelemetry distributed-tracing pipeline, introduced Canary deployment via Argo Rollouts, and handled real soft-launch traffic on the stack without incident.

9-1. End-to-End Distributed Tracing Pipeline (OTel + Tempo, 2026.06 ~ Present)

Problem

The central observability platform had logging (EFK) and metrics (kube-prometheus-stack) but no 'traces' axis, so there was no way to follow how a request flows between services or where latency arises.
Ahead of the production launch, a means to diagnose inter-service latency and errors was needed.

Approach

Built a distributed tracing stack on AWS EKS: OpenTelemetry Collector + Grafana Tempo (S3 storage via EKS Pod Identity — no static keys) + OpenTelemetry Operator (auto-instrumentation injects language SDKs with no app image change). The key decision was where to sample: RED metrics are generated from 100% of spans before sampling for accuracy, while tail sampling (all errors + everything over 500ms + 10% of the rest) applies only at the trace-storage step to cut storage cost.
Finally, wired Metrics ↔ Traces ↔ Logs into a 3-signal pivot in Grafana (the prior logging/metrics platform is Project 3).

Results

- Wired Metrics ↔ Traces ↔ Logs into a 3-signal pivot in Grafana, making it possible to follow how a request flows between services and where it stalls
- Separated RED metric generation (100% of spans, pre-sampling) from trace storage (tail-sampled), keeping metric accuracy while cutting storage cost
- Injected language SDKs with no app image change via auto-instrumentation, and accessed S3 with no static keys via EKS Pod Identity

---

9-2. Zero-Downtime Deployment with Argo Rollouts (Blue-Green → Canary, 2026.06 ~ Present)

Problem

The production game service had to keep serving live traffic during deploys, and needed instant rollback when something went wrong. But the Blue-Green setup in use swapped the ALB target group wholesale on promotion, so healthy targets momentarily dropped to zero and the ALB raced into 503s in that window.
What was needed was a way to control the transition so old and new safely coexist while traffic moves over.

Approach

Switched to Argo Rollouts Canary. On the same ALB target group, `maxUnavailable: 0` + add-before-remove (register the new Pod before removing the old) lets old and new briefly coexist during the switch, removing the 503 race; the game service is stateless HTTP (state externalized via JWT + Redis/TypeORM), so it stays safe even while traffic is split across old and new.
Templated the deployment strategy in the shared base-aws Helm chart so each service can opt in, and enabled it on just the one production game main service (a sibling internal service with no public traffic keeps a plain Deployment), with the controller running leader-election HA + PDB so rollouts never stall through node drain / Karpenter consolidation. A Pod loses traffic for different reasons on the way in and on the way out, so the guards are split in two.
On the way in, an ALB Pod readiness gate injected via a namespace label keeps a Pod from being counted Ready until it is registered healthy in the ALB target group, so the count `maxUnavailable: 0` holds equals the number of Pods that can actually take traffic. On the way out, the values run shortest first — ALB deregistration delay 30s → preStop 35s → terminationGracePeriodSeconds 60s — and only the .NET service, which drains its own in-flight requests on SIGTERM, gets 75s.
That said, changes that cannot have old and new running side by side (API contract changes, etc.) fall outside this strategy and are handled during scheduled maintenance windows given the game-service context.

Results

- Switched the production game service from Blue-Green to Canary, removing the 503 race where healthy ALB targets momentarily dropped to zero at promotion
- Backward-compatible patches deploy with zero downtime as old and new briefly coexist through the switch

9-3. Soft-Launch Operation (2026.07 ~ Present)

Results

- Soft launch went live 2026-07-14 (KST) — opened the new externally-published game to a limited release and handled its small-scale real traffic on the stack above with Canary deploys and 3-signal observability
- HPA · Karpenter autoscaling provisioned in preparation for the 2026 Q4 global launch (no scale-out has fired yet at soft-launch volume)
- Re-sized workload requests and limits from 14 days of measured soft-launch traffic — requests set from the observed per-pod peak so the HPA actually triggers.
  An over-sized request pins utilization below the HPA target permanently and the scale-out never fires; under the previous values it never had
- Completed the observability, deployment, and autoscaling foundation for the 2026 Q4 global launch

---

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game/blockchain startup, leading the large-scale AWS → GCP cloud migration.
Drove the transition to capture cost optimization and GCP's global network performance, completing the full service migration within a scheduled game-maintenance window with minimal downtime. Maintained dual source management — GitLab for on-premises, GitHub for production.
Also built a data-collection pipeline consolidating multi-source data (MongoDB · CloudSQL · GA · Dune) into BigQuery with auto-refreshed Google Sheets, eliminating the 2-3 hours of analysts' daily manual work.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Build-out

Contribution: Infra architecture design, migration strategy & execution lead 70% (remaining 30% = server-team application-side migration collaboration), CI/CD pipeline construction 100% 10 months (2023.03 ~ 2023.12)

Problem

AWS costs were continuously rising ($10,000+/month), driven largely by per-Pod billing for the always-on game workloads running on Fargate. The migration targeted cost reduction alongside the global load balancing needed for the multi-region game service.

Approach

Established a 3-phase migration strategy executed sequentially — Phase 1: build and validate the dev environment on GCP, Phase 2: migrate QA, Phase 3: migrate production.
Designed the network on Shared VPC (Host Project / Service Project separation) and switched GitHub Actions' GCP auth to Workload Identity Federation, replacing service-account key issuance/rotation with short-lived OIDC tokens. Migrated DNS via subdomain delegation (Hosting.kr → AWS Route53 → GCP Cloud DNS) for pre-validation, then performed the final NS swap during a single scheduled maintenance window.

For compute, re-platformed the workloads running on AWS Fargate onto GKE standard node pools: per-Pod billing loses to per-VM billing for always-on workloads, so GKE Autopilot — which bills the same way — was ruled out.
On cost and security, worked around Filestore's 2.5TB SSD minimum by self-hosting an NFS server on Compute Engine (pd-balanced 1TB) and configured Cloud Armor with region blocking + IP allow-list WAF policy.
Codified GCP infrastructure with Terraform (100% excluding IAM), then built the GitOps CI/CD pipeline combining GitLab CI and ArgoCD, standardizing environment configs with Helm Charts and establishing Git commit-based deployment history management.

Troubleshooting

- While migrating AWS ALB-based routing to a GCP Global LB, traffic-routing issues arose because AWS ALB fails open and still passes traffic when every target is unhealthy, whereas GCP LB blocks it.
  Analyzed the GCP NEG (Network Endpoint Group) structure and reconfigured health checks/backends for GKE workloads
- GKE Ingress + BackendConfig health check returned 503 when the configured requestPath had no response.
  Resolved by guaranteeing the health-check endpoint response and aligning ManagedCertificate / FrontendConfig to restore external ingress traffic
- Game client file downloads failed after the Cloud CDN migration due to a missing HTTP → HTTPS redirect.
  Resolved by splitting the URL Map into HTTP / HTTPS variants (`default_url_redirect.https_redirect=true`) and mapping the HTTP forwarding rule to a dedicated port-80 target HTTP proxy
- After delegating DNS to GCP Cloud DNS, the domain was not automatically registered in Google Search Console, breaking search visibility and domain-ownership verification.
  Resolved by adding the Search Console verification TXT record to Cloud DNS to verify ownership, then re-registering the domain

Results

- Designed and executed the AWS → GCP migration for a 50% cloud cost reduction ($10,000 → $5,000/month), ~$60,000 annual savings — re-platformed workloads running on Fargate onto GKE standard node pools, trading per-Pod billing for per-VM billing (Autopilot was ruled out for the same reason), while also securing an MSP reseller discount
- Completed full service migration via the 3-phase strategy.
  Minimized downtime with resource-copy based migration, and executed the final DNS cutover during an approximately 2-hour scheduled maintenance window aligned with game service operations
- Codified the entire GCP infrastructure as Terraform IaC with 100% deployment history traceable through Git commits
- Eliminated GCP service-account key management overhead and key-leak risk by adopting Workload Identity Federation for GitHub Actions, transitioning to a short-lived token security model

---

Project 2: Game Data Collection Automation System

Contribution 100% (Python script development & operations) 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, costing analysts 2-3 hours daily and causing frequent data gaps from human error.

Approach

Built a GCP-based data pipeline that loads multi-source data (MongoDB · CloudSQL · Google Analytics · Dune) into BigQuery and auto-refreshes analyst-facing Google Sheets. Per source, MongoDB was ingested via a Dataflow Flex Template ETL, CloudSQL via BigQuery's direct connection, and GA · Dune via their own APIs (Dune with a separate API key), all normalized into a consistent index schema.
Python Cloud Functions push BigQuery query results into Google Sheets via the Sheets API; Cloud Scheduler triggers them on Daily (previous-day) and Monthly (previous-month) cadences, and a separate dedup Cloud Function runs periodically to keep BigQuery data consistent. The full infrastructure (BigQuery datasets, Cloud Functions, schedulers, Cloud Storage, Artifact Registry, IAM, service accounts) is managed as Terraform IaC for reproducible environments and change tracking.

Troubleshooting

- The CloudSQL → BigQuery direct connection (federated query) failed on region mismatch and missing connection-SA permissions.
  Resolved by creating the connection in the same region and granting the connection SA the Cloud SQL Client role
- Dune API's async execute → poll → fetch flow plus rate limits caused missing results / timeouts.
  Guaranteed result retrieval via execution-status polling with backoff and moved the API key into Secret Manager
- Daily re-runs re-loaded duplicate rows into BigQuery (non-idempotent).
  Applied MERGE / ROW_NUMBER() dedup logic in the Cloud Function to make loads idempotent, keeping zero gaps/duplicates
- Loading BigQuery results into Google Sheets failed on the 10M-cell-per-sheet limit, the write quota, and the per-minute request quota (429).
  Worked around the cell limit with chunked values.batchUpdate writes and sheet splitting, and stayed within quota via call pacing/throttling, batching, and exponential backoff

Results

- 95% data collection automation (eliminated 2-3h daily manual work)
- 0% data gap rate through automation eliminating human errors
- Terraform IaC for the full infra enabled rebuilding the same pipeline in another GCP project within 30 minutes, with 100% change history in Git

Project 3: GPU Infrastructure Standardization & Scale-out for AI Workloads (Windows → Linux · Automated Setup · On-prem / GCP / External IDC)

Contribution 100% (Infra design, build & operations) 6 months (2024.02 ~ 2024.07)

Problem

The in-house GPU server ran Windows, so engineers hand-configured the runtime every time they wanted to run Stable Diffusion-family image-generation models.
Matching driver, CUDA, and framework versions by hand made the environment hard to reproduce — the hardware existed, but getting to work on it took time.

Approach

Moved the in-house GPU server from Windows to Linux (Ubuntu 22.04) and replaced the hand-configuration with a setup script. The script installs and verifies the driver stack (nvidia-driver-535 · CUDA 11.5 · cuDNN 8.6) and nvidia-docker, so whoever runs it gets the same environment, and the image-generation workloads (Stable Diffusion WebUI · kohya_ss for LoRA training · ComfyUI) sit on top as a standard configuration.
Where in-house hardware ran short, compute expanded to the cloud: a GCP VM with two NVIDIA A100 40GB cards (`a2-highgpu-2g`) provisioned as Terraform IaC, cleared through per-region/zone A100 availability checks and a GPU quota increase, then attached to the Shared VPC subnet built during the migration so the network standard held.
As demand grew further, another 20 GPU servers went into an external IDC. The setup script written for the in-house server was reused as-is, so all 20 came up through the same procedure instead of being configured one at a time. Training needs VRAM and a multi-GPU interconnect (NVLink) because the model and batch do not fit on one card, whereas image-generation inference is independent per request — there is no reason to bind cards together and throughput scales with machine count. So training and large-batch work run on the GCP A100s, while bulk inference runs on 20 consumer RTX 40-series servers at the IDC.
A new publishing deal signed afterwards kept the demand coming, and I went on hardening this GPU environment together with the publishers right up to my departure.

Results

- Replaced hand-configuration on Windows with Linux plus a setup script guaranteeing a reproducible environment — the same script later provisioned all 20 external-IDC servers, turning a one-box procedure into the standard build path for the whole GPU fleet
- Managed the GPU server as Terraform code rather than a hand-built box, standardizing rebuild and change tracking — folded into the existing GCP IaC repository
- Scaled GPU capacity from a single in-house box to 10 on-prem boxes plus GCP A100 plus 20 servers at an external IDC — matching card tier to workload shape: training, which needs a multi-GPU interconnect, versus inference, which parallelizes per request

---

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Built PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and provided technical support.
Designed and built private cloud infrastructure for financial and public sector clients, and led training for client operations teams.

Project 1: Customized PaaS Solution for Clients

Contribution 100% (Infra design & Kubernetes cluster build) 11 months (2022.04 ~ 2023.03)

Problem

Each client had different environment and server spec requirements, making standardized solutions insufficient.

Approach

Designed Kubernetes clusters in HA (High Availability) configuration per client requirements and built virtual machine management environments with OpenStack.
Provided unified block/file/object storage through Ceph and conducted parallel infrastructure operations training for client teams.

Results

- Successfully built and delivered PaaS solutions to 5 clients
- Achieved stable operation with Kubernetes cluster HA configuration
- Ansible-based server provisioning and configuration management automation reducing per-client deployment time and achieving zero configuration drift

Project 2: Kubernetes Operations Training Program for DP

Contribution 100% (Training design & delivery) 6 months (2022.06 ~ 2022.11)

Problem

DP's operations team lacked Kubernetes and container-based infrastructure experience, anticipating difficulties in self-operation after PaaS solution adoption.

Approach

Designed a step-by-step training curriculum from Kubernetes basics to cluster operations and troubleshooting, with hands-on lab environments for practice-oriented training.
Also conducted parallel training on OpenStack and Ceph storage operations to ensure full PaaS stack operational capability.

Results

- Operated the curriculum with an average of 10 attendees per session and 4 hours per session, establishing DP's autonomous Kubernetes operations system
- Reduced client technical inquiries post-training, achieved self-incident response capability
- Standardized training materials reused for subsequent client trainings

---

Project 3: Kubernetes Certificate Auto-renewal Process Improvement

Contribution 100% (Solo execution, propagated across all client clusters) 2 weeks (2022.11)

Problem

Kubernetes cluster certificates expired annually, requiring ~5 hours of renewal work per client per year, with service outage risks upon expiration.

Approach

Analyzed the Kubernetes certificate management process and modified kubeadm settings to extend certificate validity from 1 year to 10 years.
Developed automation scripts applicable to existing clusters and deployed to all clients.

Troubleshooting

- Editing kubeadm settings only worked on kubeadm-initialized clusters, and on v1.15.x / v1.16.x a known `kubeadm alpha certs renew` bug left some certificates un-renewed. On those versions, rather than relying on the kubeadm command, I adopted a public open-source script (update-kube-cert) that backs up `/etc/kubernetes` and regenerates the certificate files directly to 10-year validity.
  Applied it together with restarts of etcd, apiserver, controller-manager, scheduler, and kubelet so it works regardless of version.

Results

- 10x certificate renewal cycle extension (annual → once per 10 years)
- ~50 hours annual operations savings across 10 clients